

Algorithmics	Student information	Date	Number of session
	UO: 307423	16/04	5
	Surname: Miranda Martín	 Escuela de Ingeniería Informática Universidad de Oviedo	
	Name: Mariana Patricia		



Activity 1. [The Ferry With Vehicles]

A report and code covering the following points must be submitted:

1. State Definition: clearly explain what the indexes in your table/data structure mean. a. Hint: given that the total sum of the lengths of the first i vehicles is known, if you know the length occupied in the port lane, is it necessary to explicitly store the starboard length in the state? No.

Let S_i = sum of lengths of the first i vehicles (total occupied length).

We define $DP[i][p]$ = True if it is possible to place the i vehicle on the port side ($0 \leq p \leq L$), without exceeding L (the total occupied length on the port side is p).

Therefore, it is not necessary to store the length being occupied on the starboard side:
 $lengthOccupiedByStarboard = S_i - p$ (implicitly)

2. Recurrence relationship: write the mathematical formulation that allows you to calculate state (i, j) from the previous states. Base cases must be included.

Base: $DP[0][0] = \text{True}$; $DP[0][p] = \text{False}$ for $p > 0$

Let v_i the length of the vehicle i

Transition for $i \geq 1$: from any p where $DP[i-1][p]$ is True

- Place on port if $p + v_i \leq L \rightarrow DP[i][p+v_i] = \text{True}$
- Place on starboard if $S_i - p \leq L \rightarrow DP[i][p] = \text{True}$

The process stops at the first i whose state $DP[i][*]$ is empty (all False) or when we finish with all the vehicles.

Algorithmics	Student information	Date	Number of session
	UO: 307423	16/04	5
	Surname: Miranda Martín		
	Name: Mariana Patricia		

3. Complexity: Analyze the time cost of the algorithm as a function of N (number of vehicles) and L (length of the ferry).

The algorithm uses a dynamic programming boolean table $DP[i][p]$, where i represents the number of vehicles considered and p the occupied length in the port lane.

The table has size $(N + 1) \times (L + 1)$ as you can see in the code, and each state is computed in constant time from previous states.

Therefore, the total time complexity is $O(N \times L)$ and the space complexity is also $O(N \times L)$, due to the DP table.

As I have achieved a solution in pseudo polynomial time, I guess the shipping company will not replace me with someone who has passed Algorithmics haha.

4. Implementation: Code in the Ferry.java class that solves the problem, passing all the test cases of the companion file.

You can see the whole implementation and how the tests pass as expected in the code uploaded to GitHub.